

MODÜL 04

Geri Yayılım (Backpropagation)

Backpropagation — Chain Rule to Autograd

Zincir Kuralı · Hesaplama Grafi · Mini Autograd · Vanishing/Exploding Gradient

Seviye	Orta-İleri — Matematiksel Türetme
Kapsam	Zincir Kuralı · Hesaplama Grafi · Gradyan Kontrolü · Mini Autograd
Framework	NumPy (sıfırdan) · TensorFlow GradientTape · PyTorch Autograd
Önceki Modül	Modül 03 — Kayıp Fonksiyonları ve Optimizasyon
Sonraki Modül	Modül 05 — Regularization

Modül Konsept Haritası

Zincir Kuralı		Hesaplama Grafi		Forward Pass
$dz/dx = dz/dy \cdot dy/dx$		Her işlem düğüm		Cache değerleri
Mini Autograd		Backward Pass		Gradyan Kontrolü
Value sınıfı		dL/dW zinciri		Sayısal $\approx$ Analitik
Vanishing Grad		Gradient Sorunları		Exploding Grad
Sigmoid derin		Çözümler		RNN patlaması

1. Öğrenme Hedefleri

- ✓ Zincir kuralını tek ve çok değişkenli fonksiyonlara adım adım uygulamak
- ✓ Hesaplama grafini çizerek forward ve backward pass akışını takip etmek
- ✓ 2 katmanlı MLP için tüm gradyanları matris formuyla türetmek
- ✓ Gradyan kontrolüyle (sayısal $\approx$ analitik) backprop implementasyonunu doğrulamak
- ✓ Sıfırdan mini autograd motoru (Value sınıfı) yazmak ve test etmek
- ✓ Vanishing gradient problemini derinlik-gradyan grafiğiyle kanıtlamak
- ✓ Exploding gradient'ı tanımak ve gradient clipping ile çözmek
- ✓ TF GradientTape ve PyTorch autograd kullanımını karşılaştırmak

2. Zincir Kuralı (Chain Rule)

Zincir kuralı, iç içe geçmiş fonksiyonların türevini hesaplamak için kullanılır. Derin öğrenmede her katman, önceki katmanın çıktısını girdi olarak alır — bu tam olarak iç içe geçmiş fonksiyon yapısıdır.

$$\text{Tek değişkenli: } dz/dx = dz/dy \cdot dy/dx$$

Çok katmanlı ağ:

$$dL/dW_1 = dL/dA_2 \cdot dA_2/dZ_2 \cdot dZ_2/dA_1 \cdot dA_1/dZ_1 \cdot dZ_1/dW_1$$

2.1 Sayısal Gradyan Doğrulaması

Analitik türevin doğruluğunu kontrol etmenin en güvenilir yolu sayısal gradyan karşılaştırmasıdır. Merkezi fark formülü:

$$f'(x) \approx [f(x+h) - f(x-h)] / (2h) \quad [h=1e-5]$$

x değeri	Analitik f'(x)	Sayısal f'(x)	Mutlak Fark
-2.0	-1.000000	-1.000000	$\sim 1e-10$
-1.0	-4.000000	-4.000000	$\sim 1e-10$
0.0	-5.000000	-5.000000	$\sim 1e-10$
1.0	2.000000	2.000000	$\sim 1e-10$
2.0	15.000000	15.000000	$\sim 1e-10$

3. Hesaplama Grafi

Hesaplama grafi, her matematiksel işlemi bir düğüm (node) ve her değişkeni bir kenar (edge) olarak temsil eder. Forward pass soldan sağa, backward pass sağdan sola akar.

Örnek: $L = (w \cdot x - y)^2$

FORWARD PASS:

$x=3$ $\rightarrow$
 $\rightarrow [\times] \rightarrow z=6 \rightarrow [-] \rightarrow e=1 \rightarrow [^2] \rightarrow L=1$
 $w=2$ $\rightarrow$ $\uparrow$
 $y=5$

BACKWARD PASS (her düğümde zincir kuralı):

$dL/dL = 1$ (başlangıç - çıkış türevi her zaman 1)
 $dL/de = 2 \cdot e = 2 \cdot 1 = 2$ (kare türevi: $d(e^2)/de = 2e$)
 $dL/dz = dL/de \cdot 1 = 2$ (çıkarma türevi: $de/dz = 1$)
 $dL/dw = dL/dz \cdot x = 2 \cdot 3 = 6$ (çarpma türevi: $dz/dw = x$)
 $dL/dx = dL/dz \cdot w = 2 \cdot 2 = 4$ (çarpma türevi: $dz/dx = w$)

4. Tam MLP Backpropagation — Matris Formu

Mimari: Giriş(n) $\rightarrow$ Gizli(h) $\rightarrow$ Çıkış(1) | Aktivasyon: Sigmoid | Kayıp: BCE

FORWARD PASS:

$Z_1 = X @ W_1 + b_1$ boyutlar: $(n, n_{in}) @ (n_{in}, h) + (1, h) = (n, h)$
 $A_1 = \sigma(Z_1)$ boyutlar: (n, h)
 $Z_2 = A_1 @ W_2 + b_2$ boyutlar: $(n, h) @ (h, 1) + (1, 1) = (n, 1)$
 $\hat{y} = A_2 = \sigma(Z_2)$ boyutlar: $(n, 1) \leftarrow$ tahmin
 $L = BCE(y, \hat{y})$ skalar $\leftarrow$ minimize edilecek

BACKWARD PASS:

$\delta_2 = (A_2 - y)/n$ [BCE+Sigmoid birleşik türevi] boyut: $(n, 1)$
 $\nabla W_2 = A_1^T @ \delta_2$ boyut: $(h, 1)$
 $\nabla b_2 = \sum \delta_2$ boyut: $(1, 1)$
 $dA_1 = \delta_2 @ W_2^T$ [hatayı geri yay] boyut: (n, h)
 $\delta_1 = dA_1 \odot A_1(1-A_1)$ [sigmoid türevi, element-wise] boyut: (n, h)
 $\nabla W_1 = X^T @ \delta_1$ boyut: (n_{in}, h)
 $\nabla b_1 = \sum \delta_1$ boyut: $(1, h)$

GÜNCELLEMELER:

$W_2 \leftarrow W_2 - \eta \cdot \nabla W_2$ | $W_1 \leftarrow W_1 - \eta \cdot \nabla W_1$ [ve aynı şekilde b_2, b_1]

5. Gradyan Kontrolü (Gradient Check)

Backprop implementasyonunu doğrulamanın standart yöntemi: analitik gradyanı sayısal gradyanla karşılaştırmak. İki sonuç birbirine çok yakınsa implementasyon doğrudur. Relatif hata $< 1e-5$ genellikle yeterlidir.

Relatif Hata: $||\text{analitik} - \text{sayısal}|| / (||\text{analitik}|| + ||\text{sayısal}||)$

Relatif Hata	Yorum	Aksiyon
$< 1e-7$	Mükemmel	Hiçbir şey yapma
$1e-7-1e-5$	İyi	Kabul edilebilir
$1e-5-1e-3$	Şüpheli	Dikkatli incele
$> 1e-3$	Yanlış implementasyon	Backprop kodunu gözden geçir

NOT: Gradyan kontrolü yalnızca implementasyon doğrulaması için kullanılır. Sayısal gradyan $O(h^2)$ hassasiyetindedir ve çok yavaştır — gerçek eğitimde kesinlikle kullanılmaz.

6. Sıfırdan Mini Autograd Motoru

Autograd (otomatik türev alma), her işlemi kaydeder ve backward() çağrısında zincir kuralını otomatik uygular. PyTorch'un torch.Tensor sınıfının temelinde tam da bu mantık yatar. Biz bunu skalar değerler için Value sınıfıyla sıfırdan uyguluyoruz.

- Adım 1: Value nesnesi oluştur — data ve grad alanları
- Adım 2: İşlem yaptığında (+, *, **, sigmoid...): yeni Value + backward fonksiyonu
- Adım 3: backward fonksiyonu: gradyanı zincir kuralıyla bir önceki düğüme ilet
- Adım 4: backward() çağrıldığında: topolojik sırayla tüm backward fonksiyonlarını çalıştır

Toplama türevi:

$$\text{out} = a + b \rightarrow da/dout = 1, db/dout = 1$$

Çarpma türevi:

$$\text{out} = a * b \rightarrow da/dout = b, db/dout = a$$

Üs türevi:

$$\text{out} = a^n \rightarrow da/dout = n * a^{(n-1)}$$

Sigmoid türevi:

$$\text{out} = \sigma(a) \rightarrow da/dout = \sigma(a) \cdot (1 - \sigma(a))$$

Demo sonucu: $L = (w \cdot x - y)^2 \rightarrow dL/dw = 6.0, dL/dx = 4.0$

7. Vanishing ve Exploding Gradient

7.1 Vanishing Gradient

Sigmoid türevinin maksimumu 0.25'tir. n katmanlı bir ağda gradyan en fazla 0.25^n boyutunda olabilir. Bu üstel küçülme, derin ağlarda ilk katmanların öğrenmesini imkansız kılar.

$$|\text{grad_ilk_katman}| \leq (0.25)^n \rightarrow n=20: \approx 9.1 \times 10^{-13}$$

Katman Sayısı	Sigmoid Gradyanı	ReLU Gradyanı	Durum
1	0.2500	1.0000	Normal
5	0.000977	1.0000	Sigmoid küçülüyor
10	9.5×10^{-7}	1.0000	Sigmoid vanishing
20	$\approx 9.1 \times 10^{-13}$	1.0000	Sigmoid tamamen yok

- Çözüm: ReLU ailesi: $z > 0$ 'da türev=1 → gradyan katman boyunca kaybolmaz
- Çözüm: Batch Normalization: aktivasyonları normalize eder → gradyan akışı düzgünleşir
- Çözüm: ResNet Skip Connections: gradyan doğrudan önceki katmanlara ulaşır
- Çözüm: Gradient Highway (Dense/DenseNet): tüm önceki çıkışlar bağlanır

7.2 Exploding Gradient

Büyük ağırlıklar ve derin ağların kombinasyonu gradyanların katlanarak büyümesine yol açar. RNN'lerde özellikle yaygındır: her adımda aynı ağırlık matrisi çarpılır. $w=1.5$ ile 20 adım: $1.5^{20} \approx 3.325$ → sayısal taşma.

Gradient Clipping (Norm Bazlı):

$\text{total_norm} = \sqrt{\sum ||g||^2}$ [tüm gradyanların L2 normu]

$\text{scale} = \text{max_norm} / \text{total_norm}$ [ölçek faktörü]

$\text{g_clipped} = g \cdot \text{scale}$ (yalnızca $\text{norm} > \text{max_norm}$ ise)

8. Framework Autograd Karşılaştırması

Özellik	NumPy (Sıfırdan)	TensorFlow	PyTorch
Gradyan takibi	Manuel (Value)	GradientTape	requires_grad=True
Backward çağırısı	value.backward()	tape.gradient(L, vars)	loss.backward()
Ağırlık güncelleme	$W -= lr * dW$	optimizer.apply(grads)	optimizer.step()
Gradyan sıfırlama	Manuel w.grad=0	Otomatik (tape)	optimizer.zero_grad()
GPU desteği	Hayır	Evet	Evet (.to(device))
Dinamik graf	Her zaman	Eager Mode (TF2)	Her zaman (define-by-run)

9. Yaygın Hatalar ve Tuzaklar

Hata 1: Cache'i saklamayı unutmak	Forward pass sırasında A1, Z1 gibi ara deęerler backward için gereklidir. Cache'e kaydetmezsen backprop'ta hesaplayamazsın.
Hata 2: Gradient sıfırlamamak (PyTorch)	PyTorch gradyanları biriktirir. Her epoch başında optimizer.zero_grad() çağrılmazsa gradyanlar toplanır ve yanlış güncelleme yapılır.
Hata 3: Boyut uyumsuzluğu	$dL/dW2 = A1.T @ dZ2$ işleminde boyutlar $(h,n) @ (n,1) = (h,1)$ olmalı. Transpoz almayı unutursan boyut hatası alırsın.
Hata 4: Sayısal gradyanı üretimde kullanmak	Sayısal gradyan $O(h^2)$ hassasiyetinde ama çok yavaştır. Sadece test/debug için kullan.
Hata 5: Exploding gradient'ı NaN kaybıyla fark etmek	Loss NaN oluyorsa büyük ihtimalle exploding gradient. Gradient clipping ekle, öğrenme hızını düşür.
Hata 6: no_grad() bağlamını inference'da kullanmamak	Test/inference sırasında torch.no_grad() kullanmazsan gereksiz bellek harcanır ve işlem yavaşlar.

10. Özet Tablo

Kavram	Formül / Kural	Amaç
Zincir Kuralı	$dL/dW = dL/dA \cdot dA/dZ \cdot dZ/dW$	Gradyanı katman katman ilet
Forward Pass	$X \rightarrow Z_1 \rightarrow A_1 \rightarrow Z_2 \rightarrow A_2$ (cache kaydet)	Tahmin üret, ara değer sakla
Backward Pass	$A_2 \rightarrow Z_2 \rightarrow A_1 \rightarrow Z_1 \rightarrow W_1$ (gradyan ilet)	Tüm ∇W hesapla
Gradyan Kontrolü	$ \text{analitik} - \text{sayısal} / (a + n) < 1e-5$	Implementasyon doğrulama
Autograd	Value._backward kaydı + topo sort	Otomatik türev alma
Vanishing Grad.	$(0.25)^n \rightarrow 0$ (Sigmoid, n katman)	Derin ağ öğrenmez
Exploding Grad.	$w^n \rightarrow \infty$ ($w > 1$, n adım)	NaN kayıp, RNN sorunu
Grad. Clipping	$g *= \text{max_norm} / \text{total_norm}$ (koşullu)	Exploding çözümü

11. Kaynaklar

- [1] Rumelhart, D.E., Hinton, G.E. & Williams, R.J. (1986). Learning Representations by Back-Propagating Errors. Nature, 323, 533-536.
- [2] Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press. Bölüm 6.5: Back-Propagation.
- [3] Pascanu, R. et al. (2013). On the Difficulty of Training Recurrent Neural Networks. ICML. (Exploding/Vanishing RNN)
- [4] Karpathy, A. (2022). micrograd — Minimal autograd engine.
<https://github.com/karpathy/micrograd>
- [5] CS231n Stanford — Backpropagation, Intuitions.
<http://cs231n.github.io/optimization-2/>
- [6] PyTorch Autograd Documentation. <https://pytorch.org/docs/stable/autograd.html>
- [7] TensorFlow GradientTape Documentation.
https://www.tensorflow.org/api_docs/python/tf/GradientTape

Sonraki Adım: Modül 05 — Regularization | Dropout · Batch Normalization · L1/L2 · Early Stopping · Data Augmentation